Proyecto de Aplicación

El proyecto del curso consiste en desarrollar una aplicación en donde se ponga en práctica, con datos reales, los conceptos aprendidos. En este primer hito usted y su equipo deberán realizar el entendimiento, preparación de los datos y la aplicación de al menos **tres** de los tipos de base de datos vistos en el curso.

Fecha de entrega: Estipulada en el programa del curso

Para ser desarrollado en grupos de 3-5 personas. La asignación la realizarán ustedes, pero si alguien del curso queda solo, la asignación la realizará el profesor. Antes de armar su grupo considere los horarios que tienen en común, evite grupos en donde la posibilidad de reunirse sea mínima. Para alimentar estas bases de datos, se podrán usar cualquier fuente pública de datos, entre las cuales se encuentran:

- https://www.kaggle.com/datasets
- https://datosabiertos.bogota.gov.co/
- https://www.datos.gov.co/

También se podrá usar datos de la compañía para la que trabajas siempre y cuando la empresa esté de acuerdo en compartirlos.

Contexto

Se deberá crear una arquitectura que implique la instalación y configuración de al menos 3 bases de datos vistas en la clase. Esta arquitectura deberá implementar al menos una capa cliente, donde se puedan aplicar técnicas de visualización de la información de las bases de datos o exponer estas bases de datos usando un API con su respectiva interface gráfica.

Reporte y código del Proyecto

Se espera que el grupo desarrolle un proyecto abordando las siguientes actividades y las documente detalladamente en un artículo utilizando la plantilla oficial de IEEE (https://www.ieee.org/conferences/publishing/templates.html). El proyecto debe incluir:

1. **Introducción:**
    - **Definición del Problema y Objetivos:** Identificación clara del problema a resolver o la aplicación a desarrollar. Justificación de su relevancia. Definición de 2-3 objetivos SMART para el proyecto. (https://asana.com/es/resources/smart-goals)
    - **Alcance:** Delimitación clara de lo que incluye y no incluye la solución propuesta.
2. **Análisis y Modelado de Datos:**
    - **Identificación y Exploración:** Descripción de los datos a almacenar (tipos, fuentes, volumen estimado, características).
    - **Modelado de Datos:** Presentación de los modelos de datos para *cada* base de datos seleccionada (ej: Diagramas ER, estructuras de documentos/JSON). Se

recomienda el uso de PlantUML para documentación como código: https://plantuml.com/
- **Justificación de BBDD y Consultas:** Análisis comparativo detallado justificando la elección de cada tipo de base de datos (al menos tres, idealmente de diferente naturaleza si aplica) en función de los datos que almacenará y los tipos de consultas/accesos requeridos por la aplicación.

3. **Diseño de la Arquitectura de la Solución:**
   - **Descripción General:** Identificación de los principales componentes (ej: cliente, API REST, bases de datos, etc.), sus responsabilidades e interconexiones.
   - **Diagramas:** Inclusión de diagramas de arquitectura relevantes, mínimo la de componentes: https://plantuml.com/component-diagram
   - **Tecnologías:** Especificación y justificación de las tecnologías clave seleccionadas (lenguajes, frameworks, BBDD, etc.).
   - **Consideraciones No Funcionales:** Discusión breve sobre cómo la arquitectura aborda escalabilidad, rendimiento y seguridad.
4. **Implementación y Despliegue:**
   - **Desarrollo:** Implementación de la solución, incluyendo:
     - Configuración de al menos tres bases de datos con roles justificados.
     - Elija al menos una:
       1. Desarrollo de una API REST (aplicación online) para la lógica de negocio y acceso a datos.
       2. Desarrollo de una ETL (aplicación batch) para el procesamiento, visualización y acceso a datos.
     - Desarrollo de una aplicación cliente (web, móvil, escritorio, dashboard, chatbot) con funcionalidades mínimas definidas (ej: CRUD principal, consultas específicas, reportes resumidos).
   - **Despliegue:** Descripción del entorno de despliegue (Docker, Docker Compose, Kubernetes, o servicios cloud específicos como Atlas, DynamoDB, etc.) y cómo se configura.
5. **Pruebas y Evaluación:**
   - **Metodología de Pruebas:** Descripción de las pruebas realizadas (funcionales, integración, opcionalmente rendimiento).
   - **Resultados:** Presentación y análisis de los resultados de las pruebas, evaluando el cumplimiento de los objetivos. Comparación del rendimiento si se realizaron pruebas específicas.
6. **Conclusiones y Trabajo Futuro:**
   - **Resumen:** Síntesis de los logros, desafíos y lecciones aprendidas.
   - **Limitaciones:** Reconocimiento de las limitaciones de la solución actual.
   - **Líneas Futuras:** Sugerencias para mejoras o funcionalidades adicionales.
7. **Referencias:** Listado de todas las fuentes citadas en formato IEEE.

Entregables:

- Documento del proyecto en formato PDF (usando plantilla IEEE).
- Repositorio de código fuente (ej: GitHub) con README.md detallado para la configuración y ejecución.

Sustentación del Proyecto

La sustentación del proyecto debe durar aproximadamente 20 minutos por grupo. Se recomienda practicar para ajustar el tiempo. Esta sustentación deberá tener la siguiente estructura.

Estructura Sugerida:

1. **Elevator Pitch Estratégico (5 minutos):** El objetivo del Elevator Pitch Estratégico es convencer a posibles interesados (como inversores, clientes o jefes) sobre la idea central del proyecto, yendo más allá de una simple descripción. Para ello, debe captar la atención identificando claramente el problema real o la oportunidad relevante que aborda; presentar de forma concisa la solución única propuesta a alto nivel; destacar las ventajas y beneficios concretos que ofrece (como eficiencia, ahorro o nuevas capacidades); y concluir con una visión del potencial futuro, sugiriendo implícitamente por qué merecería apoyo o financiación. Ver ejemplo de elevator pitch acá: https://youtu.be/uv357YzY7-k
2. **Demostración Técnica y Resultados Clave (10-15 minutos):**
    - **Objetivo:** Evidenciar el trabajo técnico realizado, las decisiones clave y los resultados obtenidos. ¡Mostrar, no solo contar!
    - **Estructura Sugerida:**
        - **(1-2 min) Arquitectura y Decisiones Clave:** Presentar visualmente la arquitectura diseñada (diagrama principal). Resaltar por qué se eligieron esas bases de datos específicas y cómo se integran para resolver el problema.
        - **(5-6 min) Demo en Vivo:** Mostrar la aplicación funcionando. Enfocarse en:
            - Flujos de usuario clave.
            - Interacción con las diferentes bases de datos (ej: crear un usuario -va a PostgreSQL-, buscar un producto -va a MongoDB-, registrar un log -va a Elastic/otra-).
            - Visualización de datos (si aplica).
        - **(2-3 min) Resultados de Evaluación y Desafíos:** Presentar los hallazgos más importantes de la sección de Pruebas y Evaluación del reporte. ¿Se cumplieron los objetivos? ¿Hubo resultados inesperados (positivos o negativos)? Mencionar brevemente los principales desafíos técnicos y cómo se superaron. Usar gráficos o métricas si es relevante.

3. **Metodología y Trabajo en Equipo (2 minutos):** Describir brevemente la Metodología de desarrollo seguida (ej: sprints ágiles, Kanban), las herramientas clave de gestión y colaboración (ej: Git, Trello, Slack, Jira) y Cómo se organizó el equipo y se gestionó la comunicación/distribución de tareas.
4. **Preguntas y Respuestas del público (5 minutos)**